AI FOR FINANCE – CIA-2

AI-Powered Project Development

FraudShield AI

AI-Powered Fraud Detection System

Project Option 3 – flags suspicious financial transactions based on unusual patterns

---

Student name : _____

Register number : _____

Class / Section : _____

Faculty : _____

Date of submission : _____

1. Problem Statement

Card and digital-payment fraud is a growing problem for banks, payment apps and their customers. Fraud is difficult to catch for three reasons:

- **It is rare.** Only about 3% of transactions in this project are fraud, so a model that always says "genuine" would be 97% accurate and still useless.
- **It keeps changing.** Fraudsters adapt, so fixed rules such as "block everything above ₹50,000" quickly become outdated.
- **Mistakes are costly both ways.** A missed fraud loses money; a false alarm blocks a genuine customer's card.

Objective: build a working application that analyses each financial transaction, gives it a fraud risk score, flags suspicious transactions based on unusual patterns, and explains why each one was flagged.

2. Solution Overview

FraudShield AI is a web application built in Python. It combines **two AI models** into one hybrid risk score from 0 to 100:

	Random Forest	Isolation Forest
Type of AI	Supervised machine learning	Unsupervised anomaly detection
Learns from	Past transactions labelled fraud / genuine	Genuine transactions only
Question it answers	How much does this look like fraud seen before?	How unusual is this compared with normal behaviour?
Strength	High accuracy on known fraud patterns	Can catch new fraud patterns

Final risk score = 0.75 × Random Forest fraud probability + 0.25 × Isolation Forest anomaly score. Each transaction is then given a risk level and a recommended action:

Risk level	Recommended action
Low	Approve automatically
Medium	Step-up verification – OTP or a call to the customer
High	Block the transaction and alert the fraud team

3. Dataset

Real bank transaction data is confidential, so a **realistic synthetic dataset** was generated: **25,000 card transactions** from **600 customers** over 90 days (July–September 2026), containing **750 fraud cases (3%)**. Fraud was injected using patterns documented in real-world card fraud:

- Amounts many times larger than the customer's usual spend
- Transactions late at night (12 am – 6 am)
- Locations far from the customer's home city, or abroad
- A new or unrecognised device
- Bursts of several transactions in a short time (card testing or spending sprees)
- High-risk merchants – gift cards, crypto exchanges, electronics

To keep the problem realistic, not every fraud shows every signal: some fraud amounts look normal, and some genuine transactions look unusual (travel, occasional big purchases).

Field	Meaning
amount	Transaction amount (₹)
customer_avg_amount	Customer's usual transaction amount, from their history
merchant_category	Groceries, Travel, Electronics, Gift Cards, Crypto Exchange …
channel	POS (card swipe), Online, ATM
hour	Hour of day (0–23)
distance_from_home_km	Distance from the customer's home city
is_foreign	1 = international transaction
is_new_device	1 = device not seen before for this customer
txn_count_24h	Customer's transactions in the last 24 hours
is_fraud	Label: 1 = confirmed fraud

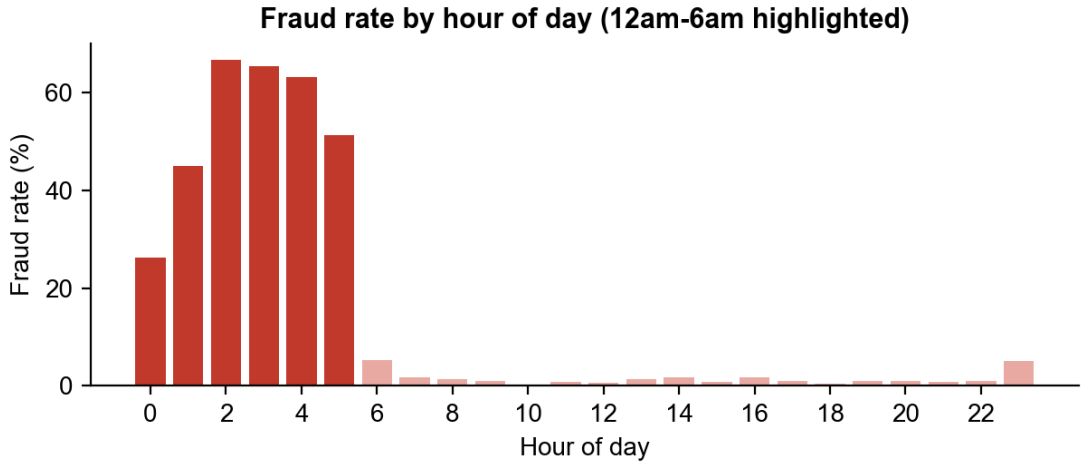


Figure 1 – Fraud is concentrated between midnight and 6 am

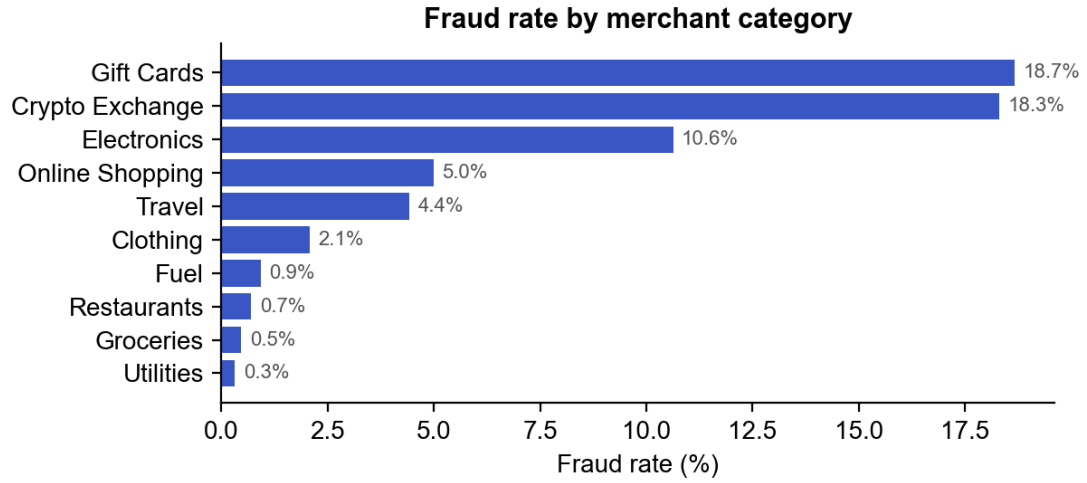


Figure 2 – Gift cards, crypto exchanges and electronics carry the highest fraud rates

4. How AI Is Used

1. **Feature engineering.** Raw fields are turned into behavioural signals. The most important one is **amount ÷ customer's usual spend**, which judges each transaction against that customer's own behaviour instead of a fixed limit. Other signals: a night-time flag, distance from home and transaction velocity (count in 24 hours).
2. **Random Forest classifier.** 300 decision trees vote on whether a transaction is fraud. Class weighting (balanced_subsample) makes the model pay attention to the rare fraud class.
3. **Isolation Forest.** Trained only on genuine transactions; it isolates points that are easy to separate from the normal crowd and gives them a high anomaly score.
4. **Threshold tuning.** The alert threshold (52.4 / 100) is chosen on a separate validation set to maximise the F1 score – balancing fraud caught against false alarms. The test set is never used for tuning.
5. **Explainable AI.** Every decision is explained in two ways: (a) plain-language red flags, e.g. "Amount is 28× the customer's usual spend", "4,200 km from home"; and (b) **Shapley values** from cooperative game theory (the idea behind SHAP), computed over all 256 what-if combinations of the 8 factors, showing exactly how much each factor raised or lowered the fraud probability.
6. **AI tools used in development.** The project was designed and coded with the help of an AI coding assistant (Claude Code), used to plan the architecture, write and debug the code, and test the web application in a browser.

5. Results

The data was split 75% for training and 25% for testing. All results below are on the **6,250 test transactions the models never saw during training**.

Model	Precision	Recall	F1 score	ROC AUC
Random Forest alone	0.901	0.872	0.886	0.998
Isolation Forest alone	0.925	0.797	0.856	0.995
Hybrid (final system)	**0.906**	**0.877**	**0.891**	**0.998**

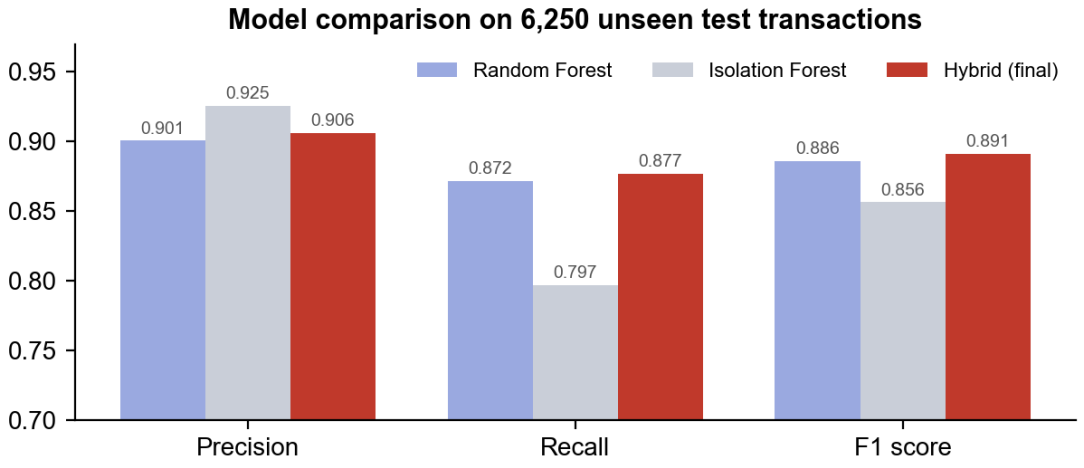


Figure 3 – The hybrid model gives the best balance (highest F1 score)

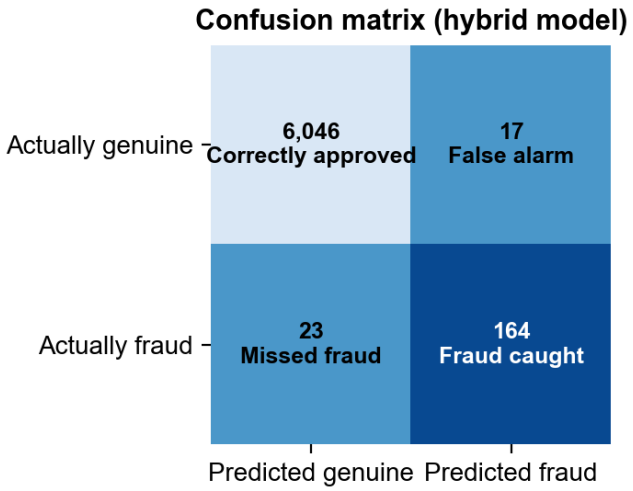


Figure 4 – Confusion matrix of the hybrid model on the test set

- **87.7%** of fraud cases were caught (recall).
- **90.6%** of alerts were real fraud (precision), so investigators do not waste time.
- **96.5%** of the money involved in fraud was caught – the large frauds are the easiest to detect.
- Only **0.3%** of genuine customers were wrongly flagged.
- On a separate batch of 1,000 brand-new transactions (the app's sample file) the system caught **28 of the 30** real frauds (93%), and 82% of its alerts were correct.

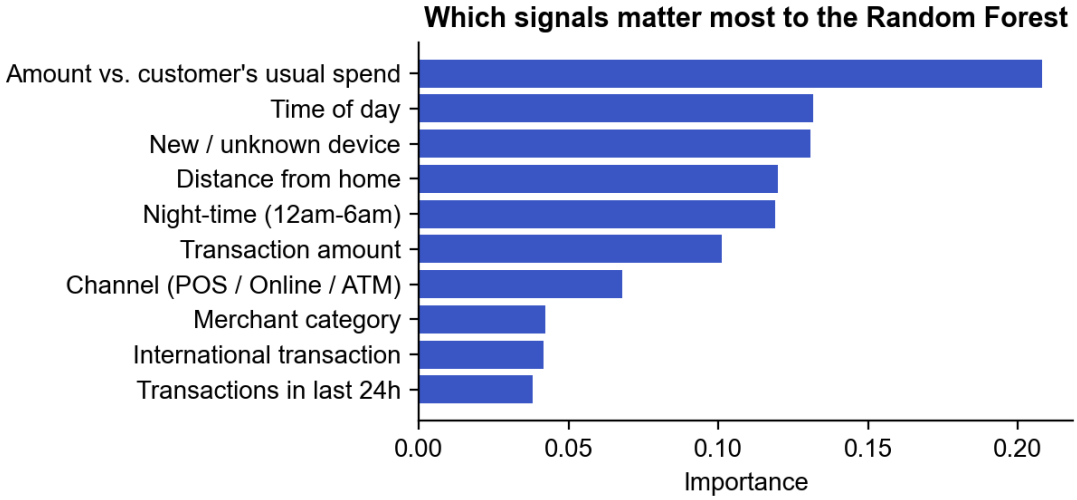


Figure 5 – Signals that matter most to the model

Note: the data is synthetic, so real-world accuracy would be lower – real data is noisier and fraudsters actively try to look normal.

6. Features of the Application

Page	What it does
Dashboard	KPIs (transactions, fraud cases, money lost) and charts of fraud patterns by hour, merchant, amount and red flag

Page	What it does
Check a Transaction	Enter one transaction and instantly get a risk gauge, a decision (approve / verify / block), red-flag reasons and a Shapley explanation chart. Normal, borderline and suspicious examples are one click away.
Batch Scan (CSV)	Upload a CSV of transactions (or use the 1,000-row sample). Every row is scored, sorted by risk with reasons, and the results can be downloaded.
Model Performance	Confusion matrix, ROC curve, comparison of the three models and feature importance
How the AI Works	Plain-language explanation of the approach and real-world uses

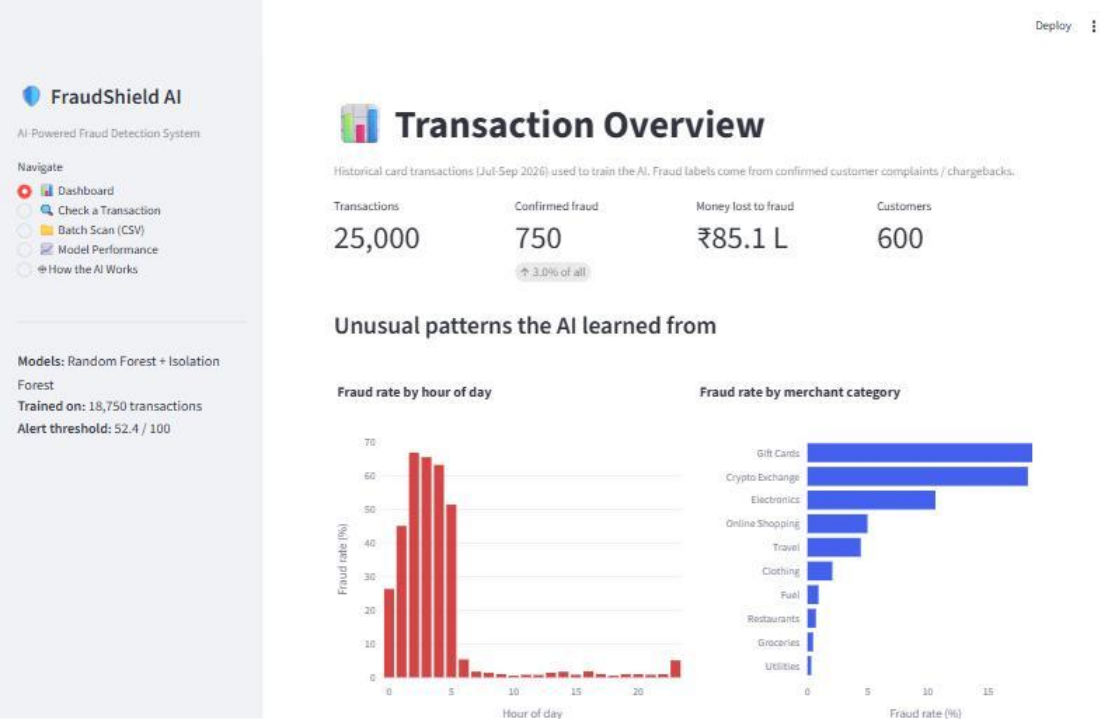


Figure 6 – Dashboard page of the FraudShield AI web application

7. Benefits and Practical Applications

- **Banks and card networks** – score card payments in real time, before they are approved.
- **UPI, wallet and e-commerce platforms** – detect account takeover and payment fraud at checkout.
- **Insurance and corporate finance** – flag unusual claims or expense reimbursements for audit.
- **Benefits** – less money lost, fewer genuine customers blocked than with fixed rules, faster investigations because alerts are ranked and explained, and easier regulatory compliance because each decision is explainable.

8. Limitations and Future Work

- Retrain on real, anonymised bank data and refresh the model regularly as fraud patterns change.
- Add richer signals – merchant reputation, IP address and device fingerprint, customer spending network.

- Try gradient boosting (XGBoost / LightGBM) and deep-learning sequence models.
- Use investigator feedback to keep improving the model (active learning).
- Deploy the scoring engine as an API connected to a live payment system.

9. Technology Stack and Project Files

Technology: Python 3.10 · scikit-learn (Random Forest, Isolation Forest) · pandas / NumPy · Streamlit (web interface) · Plotly (interactive charts) · joblib (model saving).

File	Purpose
app.py	Streamlit web application (user interface)
fraud_model.py	AI core – features, hybrid model, training, evaluation, explanations
data_generator.py	Builds the synthetic transaction dataset
data/transactions.csv	25,000 labelled transactions (training and testing)
data/sample_new_transactions.csv	1,000 unlabelled new transactions for the Batch Scan demo
models/	Saved trained model and evaluation metrics
requirements.txt / README.md	Packages needed and how to run the project

How to run: pip install -r requirements.txt, then python -m streamlit run app.py, and open http://localhost:8501 in a browser.

10. Conclusion

FraudShield AI shows how artificial intelligence can solve a real financial problem. By combining a supervised Random Forest with an unsupervised Isolation Forest, the system catches 87.7% of fraud while keeping false alarms to 0.3% of genuine customers, and it explains every decision in plain language. The same approach is used by banks and payment companies to protect customers in real time.